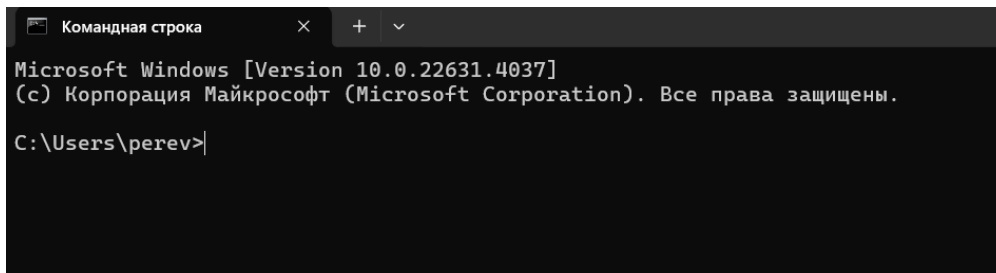


Возможности системы контроля версий (Git).

Git – это консольная утилита, для работы с ним есть, как и отдельным приложения с графическим интерфейсом (GitHub Desktop, GitKraken и т.д.), так и плагины для различных сред разработки (IDE). Сейчас я рассмотрю консольные команды, но в приложениях будут такие же названия для действий.

Консольные команды Git.

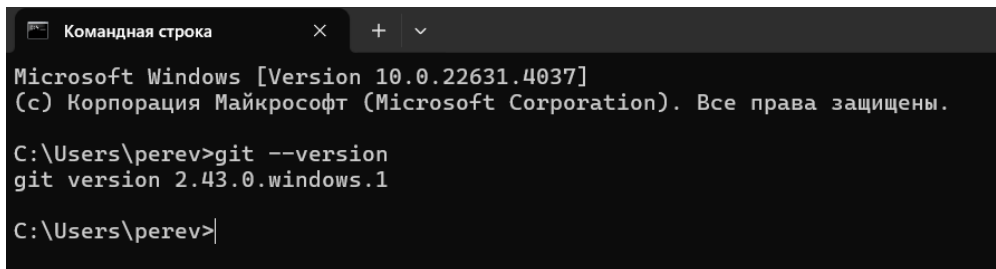
Чтобы начать работу в git, вам нужно открыть терминал или командную строку или воспользоваться встроенным терминалом среды разработки.



```
Командная строка
Microsoft Windows [Version 10.0.22631.4037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\perev>
```

Чтобы проверить версию Git, используйте команду «git --version».



```
Командная строка
Microsoft Windows [Version 10.0.22631.4037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

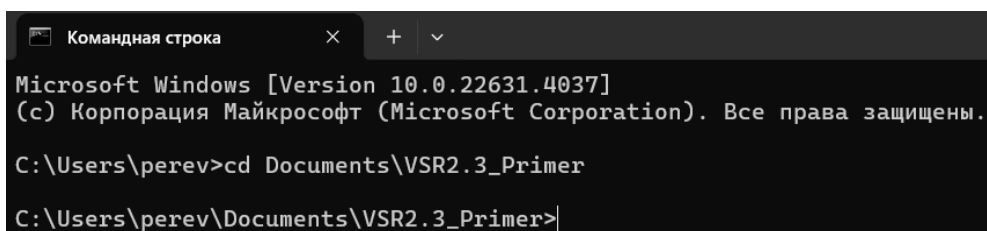
C:\Users\perev>git --version
git version 2.43.0.windows.1

C:\Users\perev>
```

Чтобы просмотреть все возможности Git, используйте команду «git --help». Она выведет весь список команд и краткое описание к ним.

Инициализация проекта.

Далее рассмотрим работу с проектами. Для этого вам понадобится папка с проектом. Для того чтобы связать наш проект с системой контроля версий нам необходимо в консоли перейти в данную папку (или открыть встроенный терминал в IDE, где уже будет нужный путь).



```
Командная строка
Microsoft Windows [Version 10.0.22631.4037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\perev>cd Documents\VSR2.3_Primer
C:\Users\perev\Documents\VSR2.3_Primer>
```

Работу с папками в консоли можно посмотреть работу ИСП1.2 (Тренировка) второго курса <https://github.com/EPereverza/Herzen>.

Встроенный терминал в VS code (рис.1 красный цвет). Также в IDE есть встроенный плагин Git, который позволяет делать все действия без консольных команд (рис.1 зеленый цвет).

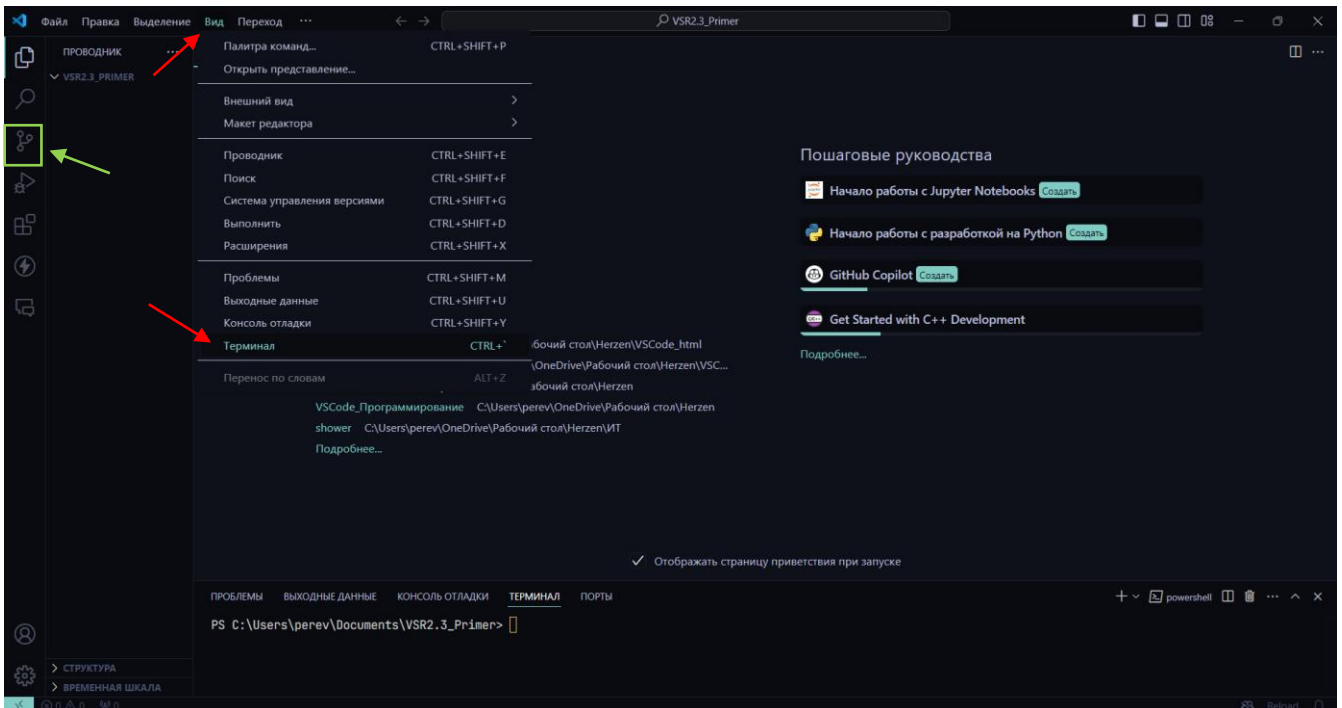


Рис.1 VS code

Первым делом нам нужно проинициализировать репозиторий командой «git init». В этот момент в папке проекта появилась скрытая папка «.git».

```
Командная строка
Microsoft Windows [Version 10.0.22631.4037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\perev>cd Documents\VSR2.3_Primer

C:\Users\perev\Documents\VSR2.3_Primer>git init
Initialized empty Git repository in C:/Users/perev/Documents/VSR2.3_Primer/.git/

C:\Users\perev\Documents\VSR2.3_Primer>|
```

Появилось сообщение о том, что проинициализирован пустой репозиторий.

Работа с файлами.

Создание файла. Создадим какой-нибудь файл в репозитории. Для того чтобы выяснить, что произошло обратимся к команде «git status».

```
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.html

nothing added to commit but untracked files present (use "git add" to track)
```

Название ветки, в которой находимся.
Коммит (изменения).
Новые файлы.

Рис. 2 git status

На рис. 2 мы видим, что после создания файла, он отобразился как не отслеживаемый файл (новый файл). Чтобы Git мог с ним работать и запоминать изменения данного файла, нужно прописать команду «git add <file>». Если у вас не один, а несколько новых файлов можно прописать «git add .», данная команда добавит сразу все файлы.

```
C:\Users\perev\Documents\VSR2.3_Primer>git add index.html

C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

Теперь у нас есть изменения (commit), за которым следит Git. Зеленым цветом обозначаются новые файлы в коммите. Красным цветом файлы, которые были добавлены, но Git “не следит” за их изменениями.

Удаление файла. Если вам потребовалось удалить файл для отслеживания, можно воспользоваться командой «git rm --cached <file>».

```
C:\Users\perev\Documents\VSR2.3_Primer>git rm --cached index.html
rm 'index.html'

C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

Я предварительно добавил index.html обратно и дополнительно добавил ещё один новый файл.

Изменение файла. Если вы внесете изменения в отслеживаемый файл, он будет подписан как modified. Но перед тем как его зафиксировать (сделать коммит), его снова требуется добавить командой «git add <file>».

```
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
        new file:   styles.css

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

C:\Users\perev\Documents\VSR2.3_Primer>git add .

C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
        new file:   styles.css
```

Важно запомнить, что если мы вносим какие-то изменения в файлы при работе с Git, нам необходимо все сохранить.

Коммит (commit). Коммит – это некое сохранение данной версии проекта, для того чтобы с этим было возможно работать в будущем.

Чтобы осуществить commit есть команда «git commit -m “описание коммита”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git commit -m "first commit"
[master (root-commit) 66346cf] first commit
2 files changed, 14 insertions(+)
create mode 100644 index.html
create mode 100644 styles.css
```

Рис. 3 commit

На рис. 3 мы видим: название ветки, в которой находимся (master); Хеш (hash) или он же уникальный код коммита (66346cf); комментарий к коммиту (first commit); кол-во измененный файлов (2 files changed); кол-во вставок (14 insertions); добавленные файлы index.html и styles.css.

Благодаря уникальному коду коммита в дальнейшем к нему можно будет возвращаться.

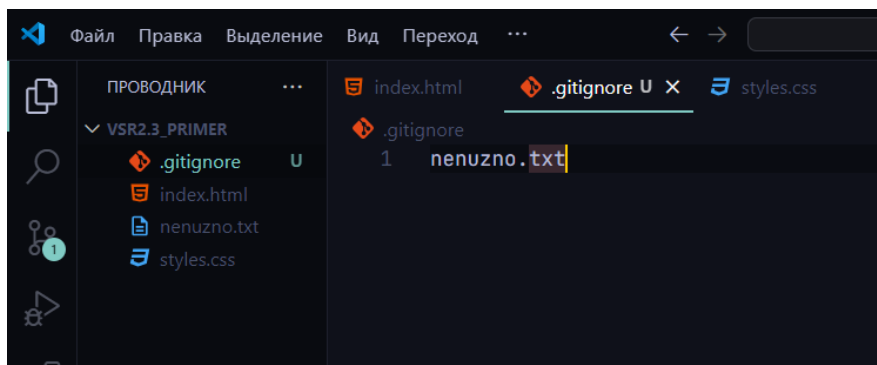
Git игнор (.gitignore).

Если вам требуется чтобы Git не видел некоторые файлы в проекте, вам пригодится «.gitignore». Допустим у вас есть какие-нибудь файл с заметками, Git его будет видеть.

```
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        nenuzho.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Чтобы прописать файл «.gitignore» нужно добавить его в корень проекта и в самом файле прописать названия файлов, которые необходимо игнорировать.



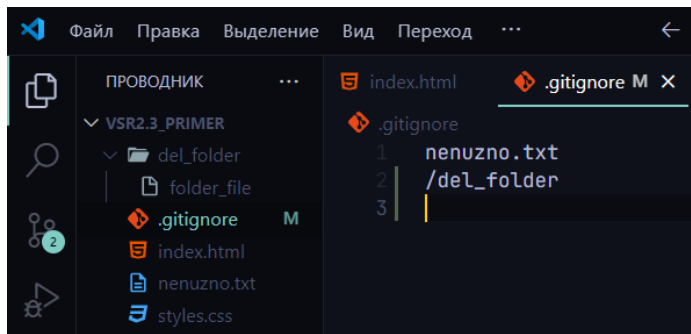
После создания «.gitignore» и добавления его для отслеживания, Git перестал видеть ненужный файл.

```
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\perev\Documents\VSR2.3_Primer>git add .
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
```

Каждый файл, который требуется проигнорировать, пишется с новой строки. Чтобы проигнорировать папку с фалами нужно написать «/”название папки”».



```
C:\Users\perev\Documents\VSR2.3_Primer>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   .gitignore
```

Работа с ветками.

Просмотр веток. Команда «git branch».

```
C:\Users\perev\Documents\VSR2.3_Primer>git branch
* master
```

Создание веток. Команда «git branch “название ветки”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git branch develop
C:\Users\perev\Documents\VSR2.3_Primer>git branch
develop
* master
C:\Users\perev\Documents\VSR2.3_Primer>
```

Удаление ветки. Команда «git branch -d “название ветки”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git branch -d develop
Deleted branch develop (was 9564b44).
C:\Users\perev\Documents\VSR2.3_Primer>git branch
* master
```

Переключение между ветками. Команда «git checkout “название ветки”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git branch
develop
* master

C:\Users\perev\Documents\VSR2.3_Primer>git checkout develop
Switched to branch 'develop'

C:\Users\perev\Documents\VSR2.3_Primer>git branch
* develop
  master
```

Создать ветку и сразу на неё переключиться. Команда «git checkout -b “название новой ветки”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git checkout -b feature
Switched to a new branch 'feature'

C:\Users\perev\Documents\VSR2.3_Primer>git branch
  develop
* feature
  master
```

Теперь для рассмотрения дальнейших функций удалим ветку feature и переключимся в ветку develop. Создадим новый файл в данной ветке, добавим его в отслеживание и сделаем коммит. Получим следующий список файлов в IDE (рис. 4). Переключимся на ветку master. Мы видим, что наш созданный файл есть только в ветке develop (рис. 5).

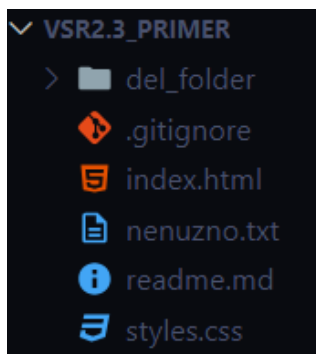


Рис. 4

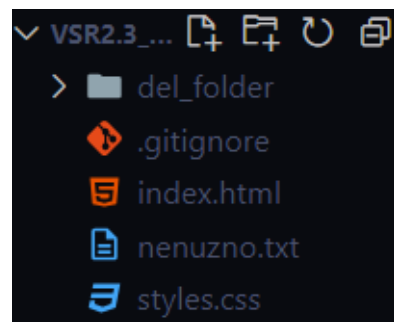


Рис. 5

Соединение веток (merge). Для этого вам нужно находиться в той ветке, к которой вы хотите соединить другую. Команда «git merge “название ветки, которую присоединяете”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git branch
  develop
* master

C:\Users\perev\Documents\VSR2.3_Primer>git merge develop
Updating 9564b44..dfbb2bd
Fast-forward
 readme.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 readme.md
```

После соединения веток делать коммит не требуется.

Работа с GitHub.

Привязка имени пользователя и почты. Для начала создадим репозиторий на GitHub для нашего проекта. Далее нужно связать GitHub и Git на вашем ПК, если вы это не делали ранее. Команды:

- «git config --global user.name “ваше имя пользователя”», чтобы задать имя пользователя (изменить) или «git config--global user.name», чтобы посмотреть какое сейчас имя пользователя.
- «git config --global user.email “ваша почта”», чтобы задать почту пользователя (изменить) или «git config --global user.email», чтобы посмотреть какая сейчас почта.

Привязка удаленного репозитория и локального. После ввода имени пользователя и почты, требуется сделать привязку удаленного репозитория на GitHub с нашим локальным репозиторием проекта.

Команда «git remote add origin “ссылка на удаленный репозиторий”.git»

```
C:\Users\perev\Documents\VSR2.3_Primer>git remote add origin https://github.com/EPereverza/VSR2.3.git
```

Опубликовать локальный проект на удаленный сервер (GitHub).

Команда «git push -u origin “название ветки, которую хотим добавить на GitHub”».

```
C:\Users\perev\Documents\VSR2.3_Primer>git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (10/10), 980 bytes | 980.00 KiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/EPereverza/VSR2.3.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Добавить изменения репозитория на GitHub. Допустим мы изменили какой-нибудь файл. Чтобы эти изменения сохранились на GitHub, нужно:

- Добавить изм. файл «git add “изм. файл”»,
- Сделать коммит «git commit -m “описание”»,
- Опубликовать изм. на GitHub «git push».

Клонирование репозитория GitHub через консоль.

- Создать папку под репозиторий,
- Перейти в эту папку,
- Клонирование «git clone “ссылка на репозиторий”.git».

```
C:\Users\perev\Documents>mkdir git-project
C:\Users\perev\Documents>cd git-project
C:\Users\perev\Documents\git-project>git clone https://github.com/EPereverza/VSR2.3.git
Cloning into 'VSR2.3'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 13 (delta 2), reused 13 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), done.
Resolving deltas: 100% (2/2), done.
```

Синхронизация изменений репозитория GitHub с локальным. Команда «git pull».